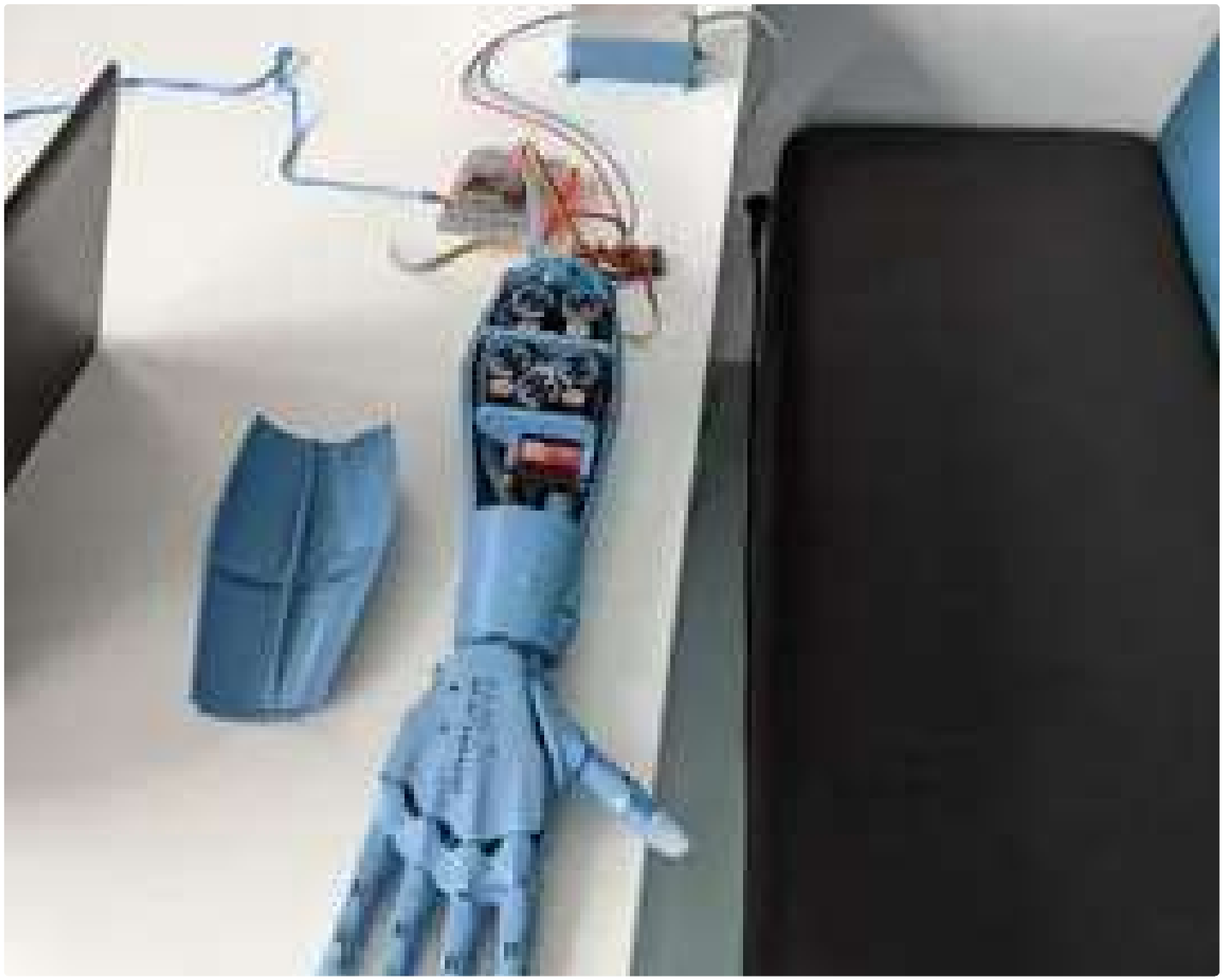
Mão biônica controlada por OpenCV

Por bloudakm em Circuitos > Robôs 20,425 221 4 Apresentou Ganhador

BY-NC-SA



Olá a todos, neste tutorial gostaria de guiá-los pelo processo de criação e, principalmente, controle de sua própria mão biônica. Essa ideia surgiu quando eu estava navegando pelo Instagram sem rumo e me deparei com um vídeo curto onde uma pessoa usava sensores MPU6050 para rastrear os movimentos da mão e exibir um modelo 3D dela em uma tela. Como eu já havia trabalhado com esse sensor, pensei que seria um projeto que eu conseguiria realizar. Além disso, adoro conectar o mundo da programação com o mundo físico real, então imaginei: por que não pegar os dados coletados e transferi-los para uma mão biônica de verdade? E assim começou. Mais tarde, decidi que uma maneira mais eficiente seria usar o OpenCV em vez do MPU6050, em parte também porque era uma forma de explorar e aprender uma nova habilidade.

Note que o design da mão foi retirado do inmoov , portanto, um enorme agradecimento a Gaël Langevin, que teve a gentileza de compartilhar seu incrível trabalho com o resto do

mundo.

Suprimentos

Mão e antebraço Inmoov i2

- impressora 3D
- equipamento de soldagem
- um pouco menos de 1 kg de filamento (PETG, ABS ou PLA)
- Linha de pesca de 3m (capaz de suportar cerca de 20kg)
- 5 molas de expansão 3/16″ x 1-3/4
- Silicone RTV Ecoflex™ 00-10

Parafusos, porcas e arruelas

- 10 parafusos de madeira de cabeça chata M2x4
- 5 parafusos de cabeça escareada M3x4mm
- 4 parafusos de madeira de cabeça chata M3x12mm
- 20 parafusos de cabeça escareada M3x12mm
- 25 parafusos de cabeça escareada M3x16mm
- 10 parafusos de cabeça escareada M3x20mm
- 35 porcas M3

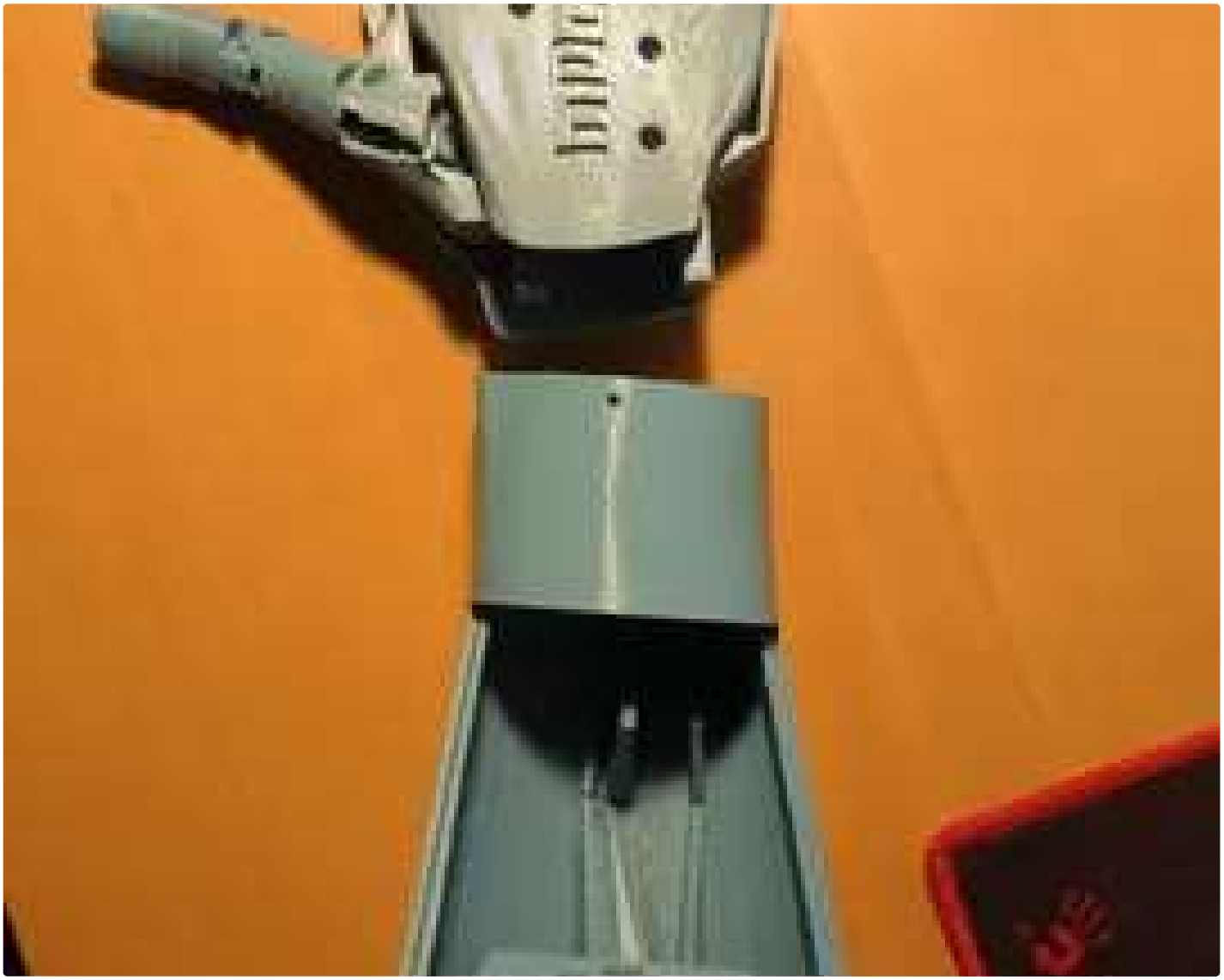
Eletrônica

- 1x Módulo de desenvolvimento ESP32 de 38 pinos
- 1 cabo micro USB
- Sensor Hall linear 5x (49E)
- 5 ímãs de disco com diâmetro de 2,5 mm x 1 mm
- 1 cabo de fita com 16 nervuras
- 5 resistores de 1k
- 5 resistores de 2k
- Servo motor 6x (JX PDI-6225MG-300)
- 1x placa de circuito impresso personalizada (opcional)
- 1 fonte de alimentação (idealmente 6V ou 5V e cerca de 100W, já que cada servo consome até 3A)

Etapa 1: Impressão 3D da mão

Eu carreguei todos os arquivos STL necessários para a mão i2 abaixo, embora eles também estejam disponíveis aqui . Ao imprimir, é melhor usar uma densidade de preenchimento um pouco maior (aproximadamente 30%) para aumentar a durabilidade das peças. Quanto ao material, a InMoov usa ABS, porém, se você não tiver a configuração necessária para imprimir ABS de forma confiável, PETG ou PLA também funcionarão.

Etapa 2: Impressão 3D do antebraço



Assim como para a mão, os arquivos necessários estão abaixo e também na biblioteca de peças STL do InMoov . Observe que a biblioteca de peças do InMoov contém arquivos para a versão original do robô InMoov. A mão é i2, portanto, você precisará apenas da parte das peças que são para o antebraço. **Outro ponto importante é que, ao imprimir o Bolt_entretoise7, você precisará apenas do parafuso e da presilha do meio** (as outras peças são para o modelo antigo da mão).

Você também pode imprimir um pequeno suporte de exibição , que eu mesmo projetei no Fusion 360.

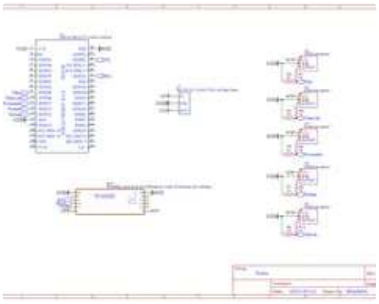
Etapa 3: Montagem

Ao montar, utilize os tutoriais da InMoov para a mão i2 e o antebraço , que são bastante detalhados e fornecem todas as informações necessárias. A parte inicial é relativamente simples, pois consiste apenas em parafusar toda a estrutura. A parte um pouco mais complicada é passar corretamente o fio de pesca para que não fique emaranhado e instalar os sensores Hall nas pontas dos dedos.

Passo 4: Pontas dos dedos de silicone

Para as pontas dos dedos, é importante usar um silicone bem macio, pois os sensores Hall têm suas incertezas e, portanto, quanto mais macio, melhor, já que o movimento do ímã em seu interior será maior e mais perceptível nos dados. Depois de colar a parte de silicone na peça impressa em 3D, use-a como ferramenta para ajustar o quanto o sensor Hall fica para fora. Após esse ajuste, recomendo fixar o sensor Hall na extremidade do dedo; caso contrário, há uma grande chance de os sensores Hall se deslocarem um pouco com o movimento dos dedos, o que afetará as medições.

Etapa 5: Circuito

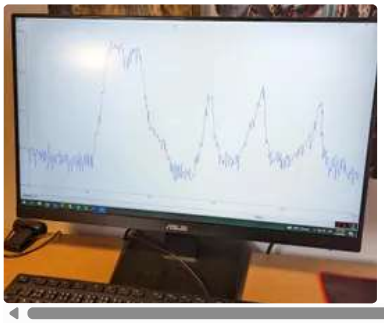


Em termos do circuito, o módulo de controle de 16 servos faz uma grande diferença, porém também apresenta algumas desvantagens. Existem duas versões diferentes deste controlador e, embora sejam quase idênticas, há uma diferença no transistor usado na proteção contra inversão de polaridade (para o capacitor). Uma versão suporta cerca de 8A, enquanto a outra suporta apenas cerca de 0,5A, o que é muito menos do que os servos consomem. Portanto, é melhor alimentar os servos sem usar o módulo controlador ou fazer o pequeno ajuste explicado neste vídeo e ter cuidado ao usar um capacitor.

Em relação aos sensores Hall, precisamos usar um divisor de tensão, pois a saída seria de 0V a 5V, o que o ESP32 não consegue ler corretamente, já que lê o ADC apenas de 0V a 3,3V.

Para todo o circuito, você pode usar uma placa de ensaio ou, de preferência, uma **placa de circuito impresso personalizada (link do GitHub para a minha versão)** .

Etapa 6: Teste



Testing Servo motors on bionic hand

Testing hall sensors on bionic hand

Como o servomotor e o sensor Hall são ligeiramente diferentes, precisamos testá-los.

O mais importante é testar os sensores Hall, pois os valores que eles medem determinarão se a mão está pressionando algo com força suficiente. Recomendo plotar os dados usando o plotter da IDE do Arduino para ver onde um valor ultrapassa a incerteza natural.

Para isso, você pode usar este trecho de código muito simples:

```
int hall = "Número do pino ao qual seu sensor Hall está conectado";

void setup() {
 Serial.begin(115200);
 pinMode(hall, INPUT);
}

void loop() {
 Serial.println(analogRead(hall));
 delay(10);
}
```

Etapa 7: Codificar

OpenCV (*executado em Python no VS Code)*

Em relação ao código que roda no PC com a webcam, temos que fazer duas coisas principais.

O primeiro passo é o rastreamento da mão e seus elementos com o OpenCV; a partir desses elementos, podemos calcular a posição de cada dedo.

A segunda coisa a fazer é enviar os dados pela porta serial para o ESP32 para que ele possa controlar os servos. Esses dados podem ser relativamente simplificados, já que não estamos enviando valores de ângulo exatos, mas sim apenas a informação de que um dedo está dobrado ou não. Assim, podemos enviar cinco números, 0 ou 1, e um símbolo no final para que possamos reconhecer posteriormente o índice de cada número.

Para começar, precisamos importar estas **bibliotecas** para o nosso código Python:

```
import cv2
import mediapipe as mp
import time
import serial
```

Em seguida, precisamos criar a classe para **processar os dados da webcam** :

```
class HandDetector():
    # Construtor da classe com parâmetros para a medição
    def __init__(self, mode=False, maxHands=1, detectionCon=0.5, trackCon=0.5):
        self.mode = mode
        self.maxHands = maxHands
        self.detectionCon = detectionCon
        self.trackCon = trackCon

        self.mpHands = mp.solutions.hands
        self.hands = self.mpHands.Hands()
        self.mpDraw = mp.solutions.drawing_utils

    # Função para encontrar e desenhar a mão
    def findHands(self, frame, draw=True):
        imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        self.results = self.hands.process(imgRGB)

        if self.results.multi_hand_landmarks:
            for handLms in self.results.multi_hand_landmarks:
                if draw:
```

```
                        self.mpDraw.draw_landmarks(frame, handLms, self.mpHands.HAND_CONNE
        return frame

    # Função para encontrar cada ponto de referência da mão e desenhar sua posição
    def findPosition(self, frame, handNo=0, draw=False):
        lmList = []

        if self.results.multi_hand_landmarks:
            myHand = self.results.multi_hand_landmarks[handNo]

            for id, lm in enumerate(myHand.landmark):
                h, w, c = frame.shape
                cx, cy = int(lm.x * w), int(lm.y * h)

                lmList.append([id, cx, cy])

                if draw and id == 0:
                    cv2.circle(frame, (cx, cy), 15, (255, 0, 255), -1)
        return lmList
```

Precisamos definir a **função principal** :

```
def main():
    # Os tempos prevTime e currentTime são usados para calcular o FPS posteriormente
    prevTime = 0
    currentTime = 0

    # Array para armazenar informações sobre a mão
    hand = [["Pulso", False], ["Indicador", False], ["Médio", False],
            ["Anelar", False], ["Polegar", False], ["Mínimo", False]]

    # Inicializando a Serial e o OpenCV
    ser = serial.Serial(port="O nome da porta à qual o ESP32 está conectado")
    # Precisei incluir "cv2.CAP_DSHOW" porque tive problemas com o carregamento da web
    cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    detector = HandDetector()

    # LOOP PRINCIPAL DO CÓDIGO #

    # Liberando os recursos alocados para o OpenCV
    cap.release()
    cv2.destroyAllWindows()

main()
```

E o **laço principal do código** :

```
while (True):
  # Encontrando as mãos e lendo a posição dos pontos de referência
  ret, frame = cap.read()
  frame = detector.findHands(frame)
  lmList = detector.findPosition(frame)

  if len(lmList) > 0:

        j = 1
        change = False
        # Loop que verifica se a ponta do dedo está abaixo da segunda ponta mais a
        for i in range(1, 6):
            if i == 1 and lmList[4][1] < lmList[3][1] and not hand[4][1]:
                # Caso seja verdadeiro, altera todos os dados necessários
                hand[4][1] = True
                change = True
                print(hand[4][0], hand[4][1])
            elif i == 1 and lmList[4][1] > lmList[3][1] and hand[4][1]:
                hand[4][1] = False
                change = True
                print(hand[4][0], hand[4][1])
            elif i != 1:
                if lmList[i*4][2] > lmList[(i*4)-2][2] e não hand[j][1]:
                    hand[j][1] = True
                    change = True
                    print(hand[j][0], hand[j][0])
                elif lmList[i*4][2] < lmList[(i*4)-2][2] e hand[j][1]:
                    hand[j][1] = False
                    change = True
                    print(hand[j][0], hand[j][0])
                if j == 3:
                    j += 2
                else:
                    j += 1

        # Se houve alguma mudança no estado da mão, este bloco de código será exec
        if change:
            msg = ""
            # Converte os valores booleanos para 0s e 1s
            for i in range(6):
                if hand[i][1]:
                    msg += "1"
```

```
                    else:
                        msg += "0"

                # Adiciona o símbolo final e envia os dados para o ESP32
                msg += '\n'
                print(msg)
                ser.write(msg.encode("Ascii"))

        # Calcula o FPS e o exibe no frame
        currentTime = time.time()
        fps = 1/(currentTime-prevTime)
        prevTime = currentTime
        cv2.putText(frame, str(int(fps)), (10, 70), cv2.FONT_HERSHEY_SIMPLEX, 3, (255,

        # Mostra o que a webcam vê em um frame
        cv2.imshow("frame", frame)

        # Se pressionarmos "q", o programa será encerrado
        se cv2.waitKey(1) & 0xFF == ord("q"):
            break
```

Código completo do OpenCV

```
import cv2
import mediapipe as mp
import time
import serial

class HandDetector():
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
        self.mode = mode
        self.maxHands = maxHands
        self.detectionCon = detectionCon
        self.trackCon = trackCon

        self.mpHands = mp.solutions.hands
        self.hands = self.mpHands.Hands()
        self.mpDraw = mp.solutions.drawing_utils

    def findHands(self, frame, draw=True):
        imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        self.results = self.hands.process(imgRGB)
```

```
        if self.results.multi_hand_landmarks:
            for handLms in self.results.multi_hand_landmarks:
                if draw:
                    self.mpDraw.draw_landmarks(frame, handLms, self.mpHands.HAND_CONNE
        return frame

    def findPosition(self, frame, handNo=0, draw=False):
        lmList = []

        if self.results.multi_hand_landmarks:
            myHand = self.results.multi_hand_landmarks[handNo]

            for id, lm in enumerate(myHand.landmark):
                h, w, c = frame.shape
                cx, cy = int(lm.x * w), int(lm.y * h)

                lmList.append([id, cx, cy])

                if draw and id == 0:
                    cv2.circle(frame, (cx, cy), 15, (255, 0, 255), -1)
        return lmList

def main():
    prevTime = 0
    currentTime = 0
    hand = [["Wrist", False], ["Indicador", False], ["Médio", False],
            ["Anelar", False], ["Polegar", False], ["Mínimo", False]]


    ser = serial.Serial(port="COM3")
    cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    detector = HandDetector()

    while (True):
        ret, frame = cap.read()
        frame = detector.findHands(frame)
        lmList = detector.findPosition(frame)

        if len(lmList) > 0:

            j = 1
            change = False
```

```
        for i in range(1, 6):
            if i == 1 and lmList[4][1] < lmList[3][1] and not hand[4][1]:
                hand[4][1] = True
                change = True
                print(hand[4][0], hand[4][1])
            elif i == 1 and lmList[4][1] > lmList[3][1] and hand[4][1]:
                mão[4][1] = False
                mudança = True
                print(mão[4][0], mão[4][1])
            elif i != 1:
                se lmList[i*4][2] > lmList[(i*4)-2][2] e não hand[j][1]:
                    hand[j][1] = True
                    change = True
                    print(hand[j][0], hand[j][0])
                elif lmList[i*4][2] < lmList[(i*4)-2][2] e hand[j][1]:
                    hand[j][1] = False
                    change = True
                    print(hand[j][0], hand[j][0])
                se j == 3:
                    j += 2
                senão:
                    j += 1

        se change:
            msg = ""
            para i em range(6):
                se hand[i][1]:
                    msg += "1"
                senão:
                    msg += "0"

            msg += '\n'
            print(msg)
            ser.write(msg.encode("Ascii"))

    currentTime = time.time()
    fps = 1/(currentTime-prevTime)
    prevTime = currentTime

    cv2.putText(frame, str(int(fps)), (10, 70), cv2.FONT_HERSHEY_SIMPLEX, 3, (255,

    cv2.imshow("frame", frame)
```

```
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

    cap.release()
    cv2.destroyAllWindows()

main()
```

ESP32 *(IDE Arduino)*

Podemos tirar proveito do fato de o ESP32 ser dual-core, pois, assim como no código para PC, também precisamos realizar duas tarefas principais.

A etapa inicial consiste no recebimento de dados do PC. Como mencionado anteriormente, os dados são basicamente uma string com um número binário de seis dígitos e um símbolo de fechamento. Além disso, como esses dados são transmitidos apenas quando ocorre uma mudança de estado, podemos atribuir imediatamente esses valores (convertidos para verdadeiro ou falso) às suas respectivas variáveis. Podemos atribuir essa tarefa ao núcleo 0, já que o loop principal é executado no núcleo 1.

O segundo ponto é, obviamente, o movimento da própria mão. Para isso, precisamos de um loop que verifique constantemente o estado das variáveis e se houve alguma mudança. Quando ocorre uma mudança, os servos são movidos linearmente em incrementos menores. Após cada incremento, precisamos verificar se a variável não mudou novamente e também medir os valores lidos pelos sensores Hall. Se os valores dos sensores Hall forem muito altos, o que significa que o ímã está muito próximo do centro do dedo, também precisamos interromper o movimento dos servos.

Inicialmente, precisaremos da **biblioteca** para o **driver do servo** e também incluiremos a biblioteca Wire para a comunicação I2C:

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
```

Depois disso, precisamos definir os **valores da duração do pulso** , que são diferentes para cada tipo de servo, então recomendo fortemente que você pesquise os valores específicos para o seu **servo** ou os teste desta forma .

```
// Velocidade de operação do meu servo (6V): 0,21 seg/60°

#define SERVOMIN "Seu valor (o meu era 70)" // Este é o número mínimo de pulsos (de um
#define SERVOMAX "Seu valor (o meu era 510)" // Este é o número máximo de pulsos (de u
#define SERVO_FREQ 50 // Servos analógicos operam com taxas de atualização de aproxima
```

Agora precisamos **definir** as demais **variáveis** que vamos usar:

```
// Inicializando o objeto do driver do servo
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

// Indicador, Médio, Anelar, Polegar, Mínimo
// "state0" é o estado da mão na webcam e "state"
// é o que está acontecendo na mão real
bool state0[6] = {false, false, false, false, false, false};
bool state[6] = {false, false, false, false, false, false};

// Variável que indica se houve alguma alteração no estado
bool change = false;

// Variáveis necessárias para ler os dados da Serial
char sData;
String state;

// Variável para o sensor Hall
// Indicador, Médio, Anelar, Polegar, Mínimo
// {pino, valor medido, valor máximo}
// TODOS OS VALORES MÁXIMOS FORAM MEDIDOS POR MIM, PORTANTO, PROVAVELMENTE NÃO SERÃO (
int hall[5][3] = {{26, 0, 2200}, {27, 0, 2400}, {14, 0, 2300}, {25, 0, 2200}, {12, 0,

// Definindo os números de índice de cada motor
int wrist = 0;
int thumb = 4;
int index = 1;
int middle = 2;
int ring = 3; // IMPORTANTE: este motor girará na direção oposta
int pinky = 5; // IMPORTANTE: este motor girará na direção oposta

// Função para calcular o PWM com base no grau desejado
int degToPwm(int degree) {
 return map(degree, 0, 320, SERVOMIN, SERVOMAX);
}

// Definindo os limites de grau usados
int deg = degToPwm(75);
int deg1 = degToPwm(95);
int deg2 = degToPwm(85);
int startDeg = degToPwm(180);
```

Em seguida, precisamos definir as **funções** que vamos usar:

```
// Inicialização da tarefa
TaskHandle_t recieveData;

// Função que lê os dados da Serial
void recieveDataCode(void * parameter) {
 for(;;) {
  // Loop que executa quando uma mensagem é enviada
  while(Serial.available()) {
   // Leitura de cada caractere
   sData = Serial.read();

   // Se o caractere for o símbolo de fim de linha, sabemos que é o fim da mensagem
   if(sData == '\n') {
    // Loop para converter os 0s e 1s da string em booleanos
    for(int i = 0; i < 6; i++) {
     state0[i] = state.substring(i, i+1).toInt();
    }

    // Redefinindo a variável temporária de estado
    state = "";
    // Indicando que houve uma mudança de estado
    change = true;
    break;
   } else { // Se o caractere não for o símbolo de fim de linha, adicionamos ao estado
    += sData;
   }
  }
  delay(10);
 }
}

// Função para mover os servos
void moveFinger(int fingerId, bool flex, int iteration) {
// Como os motores do anelar e do mindinho se movem em direções opostas
, // precisamos verificar quais motores estamos movendo
 if(fingerId != ring && fingerId != pinky) {
  // Também precisamos verificar se queremos que o dedo flexione ou estique
  if(flex) {
   // Além disso, o polegar se move um pouco menos, então também verificamos isso
   if(fingerId == thumb) {
    // Como queremos poder controlar o movimento por completo, precisamos
    // dividi-lo em partes menores
    float fPwm = SERVOMIN + (float(103)*float(iteration))/float(130);
```

```
      // Mas também precisamos garantir a conversão de volta para int, pois float
      // não seria aceito pela função pwm
      int iPwm = round(fPwm);
      pwm.setPWM(fingerId, 0, iPwm);
     } else { // Se o dedo não for o polegar, simplesmente o movemos
      pwm.setPWM(fingerId, 0, SERVOMIN + iteration);
     }
    } else { // Para o caso de retração, temos que fazer o oposto
     if(fingerId == thumb) {
      float fPwm = deg - (float(103)*float(iteration))/float(130);
      int iPwm = round(fPwm);
      pwm.setPWM(fingerId, 0, iPwm);
     } else {
      pwm.setPWM(fingerId, 0, deg1 - iteration);
     }
    }
   } else if(fingerId == ring || fingerId == pinky) {
    // No caso do dedo anelar ou mindinho, fazemos o mesmo
    if(flex) {
     pwm.setPWM(fingerId, 0, startDeg - iteration);
    } outro {
     pwm.setPWM(fingerId, 0, deg2 + iteração);
    }
   }
}
```

Agora só faltam as funções **de configuração** e **de repetição** :

```
void setup() {
// Iniciando a comunicação serial na mesma frequência do PC
 Serial.begin(9600);

// Atribuindo o pinMode a todos os pinos conectados ao sensor Hall
 for(int i = 0; i < 5; i++) {
  pinMode(hall[i][0], INPUT);
 }

// Configurando e iniciando o driver do servo
 pwm.begin();
 pwm.setOscillatorFrequency(27000000);
 pwm.setPWMFreq(SERVO_FREQ);

 delay(10);
```

```
// Fixando a tarefa criada ao núcleo 0
 xTaskCreatePinnedToCore(
  recieveDataCode,
  "recieveData",
  10000,
  NULL,
  0,
  &recieveData,
  0);
 delay(500);
void

loop() {
// Assim que houver uma mudança no estado, este bloco de código será executado
 if(change) {
  // Primeiro, percorrendo o número total de passos dos servos
  for(int i = 5; i < 135; i += 5) {

   // Em seguida, percorrendo todos os sensores Hall e lendo os valores
   for(int k = 0; k < 5; k++) {
    hall[k][1] = analogRead(hall[k][0]);
    // Se o valor medido for maior que o valor máximo, interrompemos o movimento
    if(hall[k][1] > hall[k][2]) {
     state1[k+1] = state0[k+1];
    }
   }

   // Em terceiro lugar, percorrendo todos os servomotores
   for(int j = 0; j < 6; j++) {
    if(state0[j] != state1[j]) {
     // Se o estado no PC não corresponder ao do ESP32,
     // chamamos a função para mover o respectivo dedo
     moveFinger(j, state0[j], i);
    }
   }
   // Este atraso é muito importante, pois define a velocidade dos movimentos
   delay(17);
  }

  // No final, igualamos as variáveis de estado novamente
  for(int i = 0; i < 6; i++) {
   state1[i] = state0[i];
  }
```

```
 }

 delay(100);
}
```

Código completo para ESP32

```
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

#define SERVOMIN "Seu valor"
#define SERVOMAX "Seu valor"
#define SERVO_FREQ 50

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

bool state0[6] = {false, false, false, false, false, false};
bool state1[6] = {false, false, false, false, false, false};

bool change = false;

char sData;
String state;

int hall[5][3] = {{26, 0, 2200}, {27, 0, 2400}, {14, 0, 2300}, {25, 0, 2200}, {12, 0,

int wrist = 0;
int polegar = 4;
int índice = 1;
int dedo médio = 2;
int anelar = 3;
int mindinho = 5;

int degToPwm(int grau) {
 return map(grau, 0, 320, SERVOMIN, SERVOMAX);
}

int deg = degToPwm(75);
int deg1 = degToPwm(95);
int deg2 = degToPwm(85);
int startDeg = degToPwm(180);
```

```
TaskHandle_t receberDados;

void receberCódigoDeDados(void * parâmetro) {
 for(;;) {
  while(Serial.available()) {
   sData = Serial.read();
   if(sData == '\n') {
    for(int i = 0; i < 6; i++) {
     state0[i] = state.substring(i, i+1).toInt();
    }
    estado = "";
    mudança = verdadeiro;
    interromper;
   } senão {
    estado += sData;
   }
  }
  atraso(10);
 }
}

void moverDedo(int idDedo, bool flex, int iteração) {
 se(idDedo != anelar && idDedo != mindinho) {
  se(flexionar) {
   se(idDedo == polegar) {
    float fPwm = SERVOMIN + (float(103)*float(iteração))/float(130);
    int iPwm = arredondar(fPwm);
    pwm.setPWM(idDedo, 0, iPwm);
   } senão {
    pwm.setPWM(idDedo, 0, SERVOMIN + iteração);
   }
  } else {
   if(fingerId == thumb) {
    float fPwm = deg - (float(103)*float(iteration))/float(130);
    int iPwm = round(fPwm);
    pwm.setPWM(fingerId, 0, iPwm);
   } else {
    pwm.setPWM(fingerId, 0, deg1 - iteration);
   }
  }
 } else /*if(fingerId == ring || fingerId == pinky)*/ {
  if(flex) {
```

```
    pwm.setPWM(fingerId, 0, startDeg - iteration);
   } else {
    pwm.setPWM(fingerId, 0, deg2 + iteration);
   }
  }
}

void setup() {
 Serial.begin(9600);

 for(int i = 0; i < 5; i++) {
  pinMode(hall[i][0], INPUT);
 }

 pwm.begin();
 pwm.setOscillatorFrequency(27000000);
 pwm.setPWMFreq(SERVO_FREQ);

 delay(10);

 xTaskCreatePinnedToCore(
  receiveDataCode,
  "receiveData",
  10000,
  NULL,
  0,
  &receiveData,
  0);
 delay(500);
}

void loop() {
 if(change) {
  for(int i = 5; i < 135; i += 5) {
   for(int k = 0; k < 5; k++) {
    hall[k][1] = analogRead(hall[k][0]);
    if(hall[k][1] > hall[k][2]) {
     state1[k+1] = state0[k+1];
    }
   }
   for(int j = 0; j < 6; j++) {
    if(state0[j] != state1[j]) {
     moveFinger(j, state0[j], i);
```

```
      }
     }
     delay(17);
    }

    for(int i = 0; i < 6; i++) {
     state1[i] = state0[i];
    }
   }

   delay(100);
  }
```

Passo 8: Aproveite os resultados